

VoicePilot AI

Sistema operativo de IA para contact centers

Hablas tú. El cliente te oye en inglés americano perfecto.

Hablas inglés con acento, o hablas español. El cliente oye lo mismo: inglés nativo, con tu entonación, en tiempo real, sin que aprietes nada. Encima de eso: el copilot que te sopla la respuesta correcta según el material de tu empresa, el vigilante de compliance, y el CRM que se escribe solo cuando cuelgas.

292 ms

Latencia añadida, Modo A
medido, no estimado

0 / 200

Respuestas inventadas
contra un modelo que miente

201

Tests automatizados
en tres lenguajes

100 %

Continuidad de audio
con la GPU muerta

Estado: especificación completa y aprobada (14 documentos). Cuatro módulos de ingeniería contruidos, probados y medidos. Un demo funcional. El resto, no empezado — por diseño.

Lo que este informe intenta hacer

Separar tres cosas que en la mayoría de los informes de producto van mezcladas: **lo que está construido y medido**, **lo que está especificado pero no existe**, y **lo que aprendimos construyendo que cambió el plan**. La tercera categoría es la más valiosa y la que casi nunca se escribe.

01

El negocio, en una frase

Un agente en Medellín, Bogotá o La Habana habla con un cliente en Miami. El cliente cree que habla con alguien de Phoenix. No lo sospecha en ningún momento de la llamada.

Ese es el producto. Todo lo demás — el copilot, el compliance, el CRM, el dashboard — es lo que hace que valga una suscripción mensual en vez de una curiosidad técnica. Pero **el acento es el gancho**, y es lo que decide si alguien contesta el correo.

Por qué esto es un negocio y no un truco

El mercado del BPO latinoamericano vende una cosa: minutos de agente más baratos que los de Estados Unidos. Compra ese descuento pagando una pena — tasas de conversión más bajas, porque una parte de los clientes cuelga en los primeros diez segundos al oír el acento. Nadie lo dice en voz alta y todo el mundo en la industria lo sabe.

VoicePilot quita la pena sin quitar el descuento. El mismo agente, el mismo costo por hora, y una tasa de contacto de agente local. Eso no es una mejora de eficiencia del 5 %; es la eliminación del único defecto estructural del modelo de negocio del cliente.

Dos modos, y por qué el segundo es más difícil

MODO	QUÉ HACE	LATENCIA AÑADIDA	ESTADO
Modo A	El agente habla inglés con acento. El cliente oye inglés americano nativo, con la entonación del agente.	292 ms medido	Núcleo probado
Modo B	El agente habla español . El cliente oye inglés. El agente nunca deja de hablar su idioma.	1.025 ms medido	Núcleo probado

La diferencia de latencia no es una deficiencia de ingeniería: es física del lenguaje. Convertir un acento no exige entender la frase — se puede empezar a convertir el tercer fonema mientras el sexto aún no existe. **Traducir exige entender, y entender exige esperar.** En español el verbo llega tarde; empezar a emitir en inglés antes de oírlo produce una frase que hay que retractar, y el audio ya emitido no se puede retractar.

Por eso el Modo A es el producto del MVP y el Modo B es la Fase 2. Un segundo de retardo es notable en una conversación; 292 ms no lo es.

El número que decide todo el proyecto
Prueba ciega en EE.UU., semana 8: ≥ 8 de 10 receptores no detectan que la voz es sintética. Si sale 5–7, seis semanas más de I+D sin construir producto. Si sale menos de 5, alto total y se replantea la tesis. Esta puerta está escrita en frío, ahora, precisamente para que dentro de cuatro meses no se decida por optimismo.

02

Qué está construido

Cuatro módulos completos, con tests, telemetría y criterio de salida verificado. Cero dependencias externas en los tres: el motor de audio no usa un solo crate, el núcleo de CRM no tiene `node_modules`, y el copilot no importa nada fuera de la biblioteca estándar de Python. Eso no es purismo — es lo que permite auditar cada línea y arrancar sin instalar nada.

MÓDULO	QUÉ ES	PARA QUÉ SIRVE	TESTS
Voice Engine Rust · media/voice-engine	El motor de audio en tiempo real: planificador de salida, detector de actividad de voz, monitor de salud, cliente de inferencia remota.	Es el corazón. Garantiza que sale un cuadro de audio al cliente cada 20 ms siempre , incluso con todo el plano de inteligencia caído. Y mide la latencia real de cualquier modelo que le pongas dentro — comprado o propio.	71
Contrato remoto gRPC · shared/proto	El contrato entre el plano de medios (Rust) y el de inteligencia (Python), con semántica explícita de fallo.	Permite que el modelo corra en otra máquina sin que la llamada se degrade. Probado contra un transporte simulado con inyección de fallos.	incl.
Núcleo de CRM TypeScript · shared/crm	Modelo canónico, contrato de adaptador, motor de mapeo, cola de sincronía idempotente, y el nivel L1 que funciona con cualquier CRM.	Es lo que hace cierto "se integra con el CRM que ya usas". Un solo modelo de datos; cada conector es un traductor. El L1 funciona incluso con un CRM casero sin API.	67
Copilot anclado Python · ai-services/co pilot-core	Recuperación híbrida, detección de disparadores, y las cuatro capas que impiden estructuralmente que el copilot invente.	Es el segundo número del producto: 0 respuestas inventadas . Si no está en el material de la empresa, el copilot calla.	63
Demo funcional demo/index.html	Un archivo. Consola del agente con llamada guionizada, conversión de voz en vivo real, y comparador manual ANTES/DESPUÉS.	Es la herramienta de venta. El comparador funciona en el teléfono, sin llave y sin internet, delante de un cliente.	36 navegador

Lo medido, y cómo verificarlo

Ninguno de estos números es una estimación. Todos salen de un programa que cualquiera puede correr.

QUÉ	RESULTADO	CÓMO SE COMPRUEBA
Latencia boca-a-oido, Modo A	292 ms , 99,9 % del audio convertido entregado	<code>cd media/voice-engine cargo run --release --bin latency-bench</code>
Latencia boca-a-oido, Modo B	1.025 ms , 97,5 % entregado	ídem

QUÉ	RESULTADO	CÓMO SE COMPRUEBA
Continuidad de audio bajo fallo	100 % con la GPU muerta, con 900 ms de stall, con 100 % de pérdida de paquetes, y con el enlace de inferencia caído a mitad de llamada	ídem
Inferencia remota, misma región	99,73 % entregado — indistinguible de local	<code>cargo run --release --example remote-report</code>
Inferencia remota, otra región	0,00 % entregado	ídem
Alucinaciones del copilot	0 en 200 pruebas , con un generador que miente en dos de cada tres llamadas. 0 citas cruzadas entre tenants.	<code>cd ai-services/copilot-core</code> <code>python3 -m eval.run</code>
Costo de las cuatro capas de defensa	0,5 ms p95	ídem

03

Lo que aprendimos construyendo

Esta es la sección que justifica haber construido antes de vender. Cada uno de estos hallazgos habría costado semanas o un cliente si se descubre en producción. Todos salieron de escribir el código y medirlo.

1 · La GPU en otra región no es una optimización pendiente. Es una llamada muerta.

Con el modelo de inferencia en la misma región: 99,73 % del audio convertido llega a tiempo — indistinguible de tenerlo en la misma máquina. Con el modelo en otra región: **0,00 %**. No "peor": cero. Cada cuadro llega después de su turno de reproducción y se descarta.

Esto convierte la colocación regional de GPU en un **requisito funcional del producto**, no en una decisión de infraestructura que se optimiza después. Cambia el modelo de costos y el plan de despliegue, y se descubrió en la semana 2 en vez de en el primer piloto.

2 · Sin modelo de embeddings, el copilot es inútil aunque no invente.

Recuperación solo léxica: **0 de 40 objeciones parafraseadas respondidas**. Cero. Con la mitad vectorial conectada: 12 de 40. Las frases que fallan son "eso es más de lo que quería gastar" y "no me alcanza el presupuesto" — que es exactamente como los clientes objetan. El manual dice "es muy caro"; nadie lo dice así en voz alta.

La especificación listaba BM25 y búsqueda vectorial como si fueran pares. No lo son. **El modelo de embeddings es bloqueante del módulo, no una mejora de fase posterior**. El roadmap está corregido.

3 · Tres defectos estaban en mi propia especificación, no en el código.

El banco de latencia encontró que el límite de backpressure especificado descartaba el 46 % de las llamadas sanas — porque un modelo en streaming retiene cuadros por construcción y el límite estaba escrito como si no. El monitor de salud no podía salir nunca del modo de emergencia, porque se alimentaba de cuadros que en emergencia dejan de existir. Y el presupuesto de latencia del Modo B omitía el margen de reproducción: 1.025 ms reales contra 930 ms estimados.

Se corrigió el documento, no el número.

4 · "Anclado" no es lo mismo que "relevante", y hay que decirlo.

El 16 % de lo que entrega la configuración híbrida del copilot está respaldado por sus citas **y no responde a la pregunta** — un párrafo de garantías ante una pregunta sobre reventa. La garantía del módulo es sobre el respaldo y se cumple entera: 0 de 200. La relevancia es trabajo del reranker, que aún no existe, y ese 16 % es el tamaño honesto del hueco.

5 · ReadyMode es dueño del audio, y eso decide un trimestre.

ReadyMode coloca la llamada. Si acepta un troncal SIP del cliente, nuestro plano de medios se inserta y todo lo construido sirve tal cual: un fin de semana de trabajo. Si no lo acepta, hace falta un driver de audio firmado en cada estación de trabajo: un trimestre, más revisión de seguridad corporativa en cada cliente.

Es la pregunta abierta más cara del proyecto, y se responde con una llamada a su equipo técnico. Está escrita como pregunta concreta en el documento 13.

El patrón

Cuatro de los cinco hallazgos son defectos de **especificación**, no de implementación. Una arquitectura no se valida leyéndola; se valida midiéndola. Por eso el orden fue construir el banco de latencia antes que el producto: para que los errores de diseño aparezcan cuando cuestan un día y no cuando cuestan un cliente.

04

Cómo se integra con cualquier CRM

La promesa comercial es "se monta encima del CRM que ya usas, cero migración". Esa frase es fácil de decir y cara de cumplir: hay ocho CRMs grandes y un número indeterminado de sistemas caseros sin API.

La solución es no tratarlos igual. Tres niveles, con criterio explícito de cuál merece cuál.

NIVEL	QUÉ HACE	ESFUERZO	PARA QUIÉN
L1 Superposición	La extensión de Chrome pinta VoicePilot encima. Lee qué registro está abierto leyendo la página. Escribe en los propios campos del CRM. Sin API.	1 semana por CRM	Cualquier CRM web , incluidos los caseros. El 100 % del mercado.
L2 Escritura	API oficial. Escribe llamadas, notas y disposiciones. Lectura bajo demanda.	2–3 semanas	Zoho, Pipedrive, la mayoría
L3 Bidireccional	Sincronía completa en ambos sentidos, webhooks entrantes, mapeo de campos personalizado, resolución de conflictos.	6–8 semanas	Salesforce, HubSpot

L3 en ocho CRMs es un equipo entero manteniendo APIs ajenas para siempre. L1 cubre todo el mercado con el 5 % del esfuerzo. Por eso el L1 es lo que está construido, y es lo que permite responder "sí" en vez de "está en el roadmap" cuando un cliente pregunta si funciona con el suyo.

Las dos preguntas difíciles del L1, y cómo se resolvieron

¿Qué registro está mirando el agente?

Sin API hay que deducirlo de una página que no es nuestra. La cascada va de más a menos fiable: patrón de URL, atributos `data-*`, selectores CSS servidos desde el servidor, y preguntar al agente.

Lo importante no es la cascada, es **qué pasa cuando dos escalones se contradicen: no se vota**. Una respuesta equivocada aquí no es una función que falta — es una nota de llamada escrita sobre el registro de otro cliente, y nadie se entera en semanas. Evidencia en conflicto produce un rechazo y una pregunta de un clic al agente, que sí está viendo la pantalla.

Además la confianza separa **mostrar** de **escribir**. Un selector CSS que lee texto renderizado basta para apuntar el copilot a un registro y no basta para escribir sin confirmación. Una sugerencia equivocada desperdicia una sugerencia; una escritura equivocada corrompe un pipeline.

¿La escritura entró de verdad?

Se verifica leyendo de vuelta antes de declarar éxito, porque el formulario ajeno hace cosas con el valor:

LO QUE PASA	POR QUÉ IMPORTA
React ignora un <code>.value=</code> programático sin evento de input	El campo parece lleno, el estado del framework está vacío, el agente guarda y no se guarda nada . Es el fallo L1 más común.
El CRM reformatea 3055550142 como (305) 555-0142	Una comparación ingenua declara fallida una escritura que funcionó, y la cola reintenta para siempre.

LO QUE PASA	POR QUÉ IMPORTA
El CRM trunca un resumen de 900 caracteres a 500	Corta el final , que es donde vive el compromiso de seguimiento: "llamar el viernes", "el crédito vence el 31".
El campo es de solo lectura, o no existe	Son problemas distintos para personas distintas: uno es del administrador del cliente, el otro es nuestra configuración obsoleta.

Cada campo lleva veredicto propio y solo "escrito y verificado" cuenta como éxito. Un campo malo nunca tumba el resto de la escritura.

05

Qué NO está construido

Enumerarlo con precisión vale más que cualquier lista de logros. Un informe que solo dice lo que existe no sirve para decidir nada.

PIEZA	ESTADO	QUÉ HACE FALTA
Puente SIP y SFU (LiveKit)	Bloqueado por entorno	Requiere descargar dependencias de Rust que este entorno no alcanza. El contrato y el cliente están hechos y probados contra un simulador con inyección de fallos: falta cableado, no diseño.
Modelo de conversión de voz propio	Se compra, no se construye	Corrección de estrategia deliberada. La conversión ya existe como servicio y el demo la usa de verdad. Para el MVP se compra: sale esta semana en vez de en seis meses. El modelo propio llega cuando el volumen justifique el costo por minuto.
Modelo de embeddings y reranker	Bloqueante de Fase 1	Los protocolos de inyección están listos y probados. Falta el modelo. Sin él el copilot no responde a objeciones parafraseadas.
ASR y transcripción en vivo	No empezado	Fase 1, módulo 1.4.
Ingesta de conocimiento (PDF, DOCX, URL)	No empezado	Fase 1, módulo 1.5. Debe entregar embeddings reales desde el primer día.
Motor de compliance	Especificado, no construido	Fase 1, módulo 1.7. El diseño de dos capas está escrito.
Consola del agente, dashboard, CRM nativo	No empezado	Fase 1. El sistema de diseño está definido (documento 12).
Extensión de Chrome	Especificada, no construida	La lógica difícil — detección de registro y escritura verificada — ya está construida y probada en shared/crm. Falta el envoltorio MV3.
Tenancy, auth, RBAC, auditoría	No empezado	Fase 1, módulo 1.1.

Riesgos reales

RIESGO	IMPACTO	QUÉ HACER
ReadyMode no acepta troncal SIP propio	Un trimestre en vez de un fin de semana, más revisión de seguridad corporativa en cada cliente.	Una llamada a su equipo técnico. Es la acción de mayor valor por minuto de todo el proyecto.
La prueba ciega sale por debajo de 8/10	El producto no existe tal como está planteado.	La puerta de decisión de la semana 8 ya está escrita. No se negocia después.

RIESGO	IMPACTO	QUÉ HACER
Consentimiento y divulgación de voz IA	Riesgo legal real. Varios estados de EE.UU. regulan la voz sintética en ventas, y la regulación se está endureciendo.	Documento 09. Hace falta opinión legal antes del primer piloto, no después.
Dependencia de un proveedor de conversión	El costo por minuto y la latencia quedan en manos ajenas; un cambio de precios cambia el margen.	El motor mide cualquier modelo que se le ponga dentro. El cambio de proveedor — o a modelo propio — es una sustitución, no una reescritura.

06

Próximos pasos, en orden

Ordenados por lo que desbloquea, no por lo que es agradable de construir.

#	ACCIÓN	POR QUÉ AHORA	CRITERIO DE SALIDA
1	Llamar al equipo técnico de ReadyMode y preguntar si aceptan un troncal SIP del cliente.	Decide entre dos arquitecturas cuya diferencia es un trimestre. Cuesta una llamada. Todo lo demás se planifica peor sin esa respuesta.	Respuesta por escrito, sí o no.
2	Grabar el ANTES/DESPUÉS con tu voz y cargarlo en el comparador del demo.	Es la herramienta de venta. Hasta que un cliente no lo oiga , todo esto es una descripción.	Dos audios en el teléfono, listos para reproducir.
3	Conectar un modelo de embeddings real al copilot.	Es bloqueante: sin él el copilot no responde a objeciones parafraseadas, que son la mayoría.	≥ 30 de 40 paráfrasis respondidas en eval.run.
4	Cablear LiveKit y el puente SIP en un entorno con acceso a dependencias.	Es lo único que separa el motor probado de una llamada telefónica real.	Una llamada PSTN real convertida extremo a extremo.
5	Prueba ciega en EE.UU. con 10 receptores.	La puerta de decisión. Nada de Fase 1 debe empezar antes de pasarla.	≥ 8 de 10 no detectan síntesis.
6	Opinión legal sobre divulgación y consentimiento de voz sintética.	Un piloto que hay que apagar por una carta de un fiscal general cuesta más que el piloto.	Guion de divulgación aprobado por abogado.
7	Fundamentos de plataforma: tenancy, auth, RBAC, auditoría.	Es la base de todo lo demás y no se puede retrofitear.	Suite de aislamiento entre tenants al 100 %.

El calendario, sin optimismo

FASE	QUÉ ENTREGA	CUÁNDO	PUERTA
Fase 0 Prueba de latencia	La respuesta a una sola pregunta: ¿se puede convertir voz en tiempo real con calidad de venta y latencia invisible? No se construye producto.	Semanas 1–12 <i>parcialmente cumplida</i>	Prueba ciega ≥ 8/10, semana 8
Fase 1 MVP vendible	Un call center piloto operando de verdad, todos los días.	Semanas 13–32	500+ llamadas/semana durante 4 semanas · 0 alucinaciones · –60 % de trabajo post-llamada
Fase 2 Producto	De un piloto a 20 clientes que renuevan. Modo B en producción, Salesforce, SOC 2 Tipo I.	Meses 9–14	20 clientes activos · retención > 90 % · NPS > 40
Fase 3 Enterprise	Contratos de seis cifras.	Meses 15–24	—

Principio de ejecución

Un módulo no se abandona a medias. Se termina — con tests, telemetría, documentación y criterio de salida verificado — antes de empezar el siguiente. Diez módulos al 80 % valen cero; seis al 100 % son un producto. Esa es la razón de que este informe tenga una lista corta de cosas construidas y una lista larga de cosas explícitamente no empezadas.

Cómo verificar todo esto por tu cuenta

```
cd voicepilot-ai/media/voice-engine
cargo test # 71 tests
cargo run --release --bin latency-bench # la tabla de latencia

cd ../../shared/crm
npm test # 67 tests

cd ../../ai-services/copilot-core
python3 -m unittest discover -s . -p "test_*.py" -t . # 63 tests
python3 -m eval.run # 0 alucinaciones / 200
```